

LAPORAN PRAKTIKUM
SISTEM OPERASI



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
Subhan Maulana Andrianto(2410651014)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

1. PROCESS_MANAGER

adalah bagian dari sistem operasi yang bertanggung jawab untuk mengatur semua aktivitas yang berkaitan dengan proses (program yang sedang berjalan) di dalam komputer.

Tugas Utama Process Manager :

1. Membuat dan Menghapus Proses
 - Membuat proses baru menggunakan sistem panggilan seperti fork() atau exec() (di Linux).
 - Menghapus proses setelah selesai (exit()).
2. Penjadwalan Proses (Scheduling)
 - Menentukan urutan proses mana yang akan dieksekusi CPU.
 - Menggunakan algoritma seperti Round Robin, FCFS, Priority Scheduling, dll.
3. Sinkronisasi dan Komunikasi Antarproses (IPC)
 - Mengatur proses agar bisa berkomunikasi dan berbagi data dengan aman.
 - Contohnya: pipe, message queue, shared memory, signal.
4. Manajemen Status Proses
 - Menyimpan dan memperbarui state proses (Running, Ready, Waiting, Terminated).
 - Menangani context switching (berpindah dari satu proses ke proses lain).
5. Penanganan Deadlock
 - Mendeteksi dan mengatasi kondisi ketika dua atau lebih proses saling menunggu sumber daya.

INPUT :

```
GNU nano 7.2 process_manager.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <stdbool.h>
#include <signal.h>
#include <errno.h>

#define MAX_BUF 2048

pid_t child_pids[3] = {0,0,0};

void handle_sigint(int sig) {
    printf("\n[Parent] SIGINT diterima - menghentikan semua child secara graceful...\n");
    for (int i = 0; i < 3; i++) {
        if (child_pids[i] > 0) {
            kill(child_pids[i], SIGTERM);
            printf("[Parent] Mengirim SIGTERM ke child PID %d\n", child_pids[i]);
        }
    }
    exit(0);
}

long long factorial(int n) {
    if (n <= 1) return 1;
    long long res = 1;

    for (int i = 2; i <= n; i++) res *= i;
    return res;
}

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i*i <= n; i++) if (n % i == 0) return false;
    return true;
}

int main() {
    int p2c[3][2];
    int c2p[3][2];
    struct timeval start_times[3];
    struct timeval end_time;

    printf("=== Process Manager (fork, pipe, signal) ===\n");

    for (int i = 0; i < 3; i++) {
        if (pipe(p2c[i]) == -1) { perror("pipe p2c"); exit(1); }
        if (pipe(c2p[i]) == -1) { perror("pipe c2p"); exit(1); }
    }

    int input_fact;
    int input_prime_limit;
    char input_str[1024];

    printf("Masukkan angka untuk Faktorial (Child 1): ");
    if (scanf("%d", &input_fact) != 1) { fprintf(stderr, "Input tidak valid\n"); exit(1); }

    printf("Masukkan batas bilangan prima (Child 2): ");
    if (scanf("%d", &input_prime_limit) != 1) { fprintf(stderr, "Input tidak valid\n"); exit(1); }
    getchar();
    printf("Masukkan string (Child 3): ");
    if (fgets(input_str, sizeof(input_str), stdin) == NULL) { fprintf(stderr, "Error baca string\n"); exit(1); }
    input_str[strcspn(input_str, "\n")] = 0;

    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    for (int i = 0; i < 3; i++) {
        gettimeofday(&start_times[i], NULL);
        pid_t pid = fork();

        if (pid < 0) { perror("fork"); exit(1); }

        if (pid == 0) {
            for (int j = 0; j < 3; j++) {
                if (j != i) {
                    close(p2c[j][0]); close(p2c[j][1]);
                    close(c2p[j][0]); close(c2p[j][1]);
                }
            }
            close(p2c[i][1]);
            close(c2p[i][0]);
        }
    }
}
```

```

        if (i == 0) {
            int n;
            read(p2c[i][0], &n, sizeof(n));
            long long res = factorial(n);
            char buf[MAX_BUF];
            snprintf(buf, sizeof(buf), "%lld", res);
            write(c2p[i][1], buf, strlen(buf)+1);
            exit(10);
        } else if (i == 1) {
            int limit;
            read(p2c[i][0], &limit, sizeof(limit));
            char outbuf[MAX_BUF];
            int pos = 0;
            outbuf[0] = '\0';
            pos = 0;
            for (int num = 2; num <= limit; num++) {
                if (isPrime(num)) {
                    pos += snprintf(outbuf+pos, sizeof(outbuf)-pos, "%d ", num);
                    if (pos >= (int)sizeof(outbuf)-50) break;
                }
            }
            if (pos == 0) snprintf(outbuf, sizeof(outbuf), "(tidak ada)");
            write(c2p[i][1], outbuf, strlen(outbuf)+1);
            exit(20);
        } else {
            char buf[1024];
            ssize_t r = read(p2c[i][0], buf, sizeof(buf)-1);
            if (r > 0) buf[r] = 0; else buf[0] = 0;

```

```

            int L = strlen(buf);
            for (int a = 0; a < L/2; a++) {
                char tmp = buf[a];
                buf[a] = buf[L-a-1];
                buf[L-a-1] = tmp;
            }

            write(c2p[i][1], buf, strlen(buf)+1);
            exit(30);
        }
    } else {
        child_pids[i] = pid;
    }
}

close(p2c[0][0]); close(p2c[1][0]); close(p2c[2][0]);
close(c2p[0][1]); close(c2p[1][1]); close(c2p[2][1]);

write(p2c[0][1], &input_fact, sizeof(input_fact));
write(p2c[1][1], &input_prime_limit, sizeof(input_prime_limit));
write(p2c[2][1], input_str, strlen(input_str)+1);

close(p2c[0][1]); close(p2c[1][1]); close(p2c[2][1]);

int status;
for (int i = 0; i < 3; i++) {
    pid_t w = waitpid(child_pids[i], &status, 0);
    gettimeofday(&end_time, NULL);

```

```

    double elapsed = (end_time.tv_sec - start_times[i].tv_sec) * 1000.0 +
        (end_time.tv_usec - start_times[i].tv_usec) / 1000.0;

    printf("\n[Parent] Child PID %d selesai.\n", w);
    if (WIFEXITED(status))
        printf("[Parent] Exit status: %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[Parent] Terminated by signal %d\n", WTERMSIG(status));

    printf("[Parent] Waktu eksekusi: %.3f ms\n", elapsed);

    char buf[MAX_BUF];
    ssize_t r = read(c2p[i][0], buf, sizeof(buf)-1);
    if (r > 0) { buf[r] = 0; printf("[Parent] Hasil Child %d: %s\n", i+1, buf); }
}

printf("\n[Parent] Semua child selesai. Program selesai.\n");
return 0;
}

```

OUTPUT :

```
root@DESKTOP-J9C60BQ:~# gcc -o process_manager process_manager.c
root@DESKTOP-J9C60BQ:~# ./process_manager
=== Process Manager (fork, pipe, signal) ===
Masukkan angka untuk Faktorial (Child 1): 5
Masukkan batas bilangan prima (Child 2): 20
Masukkan string (Child 3): andre

[Parent] Child PID 2906 selesai.
[Parent] Exit status: 10
[Parent] Waktu eksekusi: 1.305 ms
[Parent] Hasil Child 1: 120

[Parent] Child PID 2907 selesai.
[Parent] Exit status: 20
[Parent] Waktu eksekusi: 0.934 ms
[Parent] Hasil Child 2: 2 3 5 7 11 13 17 19

[Parent] Child PID 2908 selesai.
[Parent] Exit status: 30
[Parent] Waktu eksekusi: 0.730 ms
[Parent] Hasil Child 3: erdna

[Parent] Semua child selesai. Program selesai.
```

KESIMPULAN

Program `process_manager` berhasil mengimplementasikan komunikasi dan manajemen antar proses menggunakan `fork`, `pipe`, dan `signal`. Setiap proses anak menjalankan tugas spesifik (faktorial, bilangan prima, dan pembalik string) secara mandiri, dan hasilnya berhasil dikirim serta ditampilkan oleh proses induk. Program berjalan dengan baik dan menunjukkan cara kerja proses paralel di sistem operasi Linux.

Berikut penjelasannya :

Child 1 (PID 2906)

- Tugas: Menghitung faktorial dari angka 5.
- Hasil: $5! = 120$
- Exit status: 10
- Waktu eksekusi: 1.305 ms

Child 2 (PID 2907)

- Tugas: Menampilkan bilangan prima sampai batas 20.
- Hasil: 2 3 5 7 11 13 17 19
- Exit status: 20
- Waktu eksekusi: 0.934 ms

Child 3 (PID 2908)

- Tugas: Membalik string yang dimasukkan, yaitu "andre".
- Hasil: "erdna"
- Exit status: 30
- Waktu eksekusi: 0.730 ms

2. FILE MONITOR

File monitor adalah program atau sistem yang digunakan untuk mengamati (memantau) perubahan yang terjadi pada file atau direktori di suatu sistem komputer secara real-time.

Perubahan yang dimaksud bisa meliputi:

1. Penambahan file baru
2. Penghapusan file
3. Perubahan isi atau ukuran file
4. Perubahan izin akses (permissions)
5. Pemindahan (rename/move) file

INPUT :

```
GNU nano 7.2 file_monitor.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <string.h>

#define EVENT_BUF_LEN (1024 * (sizeof(struct inotify_event) + 16))
#define LOG_FILE "file_monitor.log"

int inotify_fd;
int watch_fd;
int log_fd;
int running = 1;

void sigterm_handler(int sig) {
    (void)sig;
    write(STDOUT_FILENO, "\nSIGTERM diterima, menghentikan monitor...\n", 43);
    running = 0;
}

int has_valid_extension(const char *filename) {
    const char *dot = strrchr(filename, '.');
    if (!dot) return 0;
    if (strcmp(dot, ".txt") == 0 || strcmp(dot, ".log") == 0)
        return 1;
    return 0;
}

void log_event(const char *event_type, const char *filename, pid_t pid) {
    time_t t = time(NULL);
    struct tm *tm_info = localtime(&t);
    char timestamp[32];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", tm_info);

    char buffer[512];
    int len = snprintf(buffer, sizeof(buffer), "[%s] [%s] [%s] [PID:%d]\n",
                      timestamp, event_type, filename, pid);
    write(log_fd, buffer, len);
}

int main() {
    char dir_path[256];
    printf("=== File Monitor (inotify) ===\n");
    printf("Masukkan path direktori untuk dipantau (misal: /tmp/monitor_dir): ");
    scanf("%255s", dir_path);

    signal(SIGTERM, sigterm_handler);

    inotify_fd = inotify_init();
    if (inotify_fd < 0) {
        perror("inotify_init");
        exit(EXIT_FAILURE);
    }
}
```

```

watch_fd = inotify_add_watch(inotify_fd, dir_path, IN_CREATE | IN_MODIFY | IN_DELETE);
if (watch_fd < 0) {
    perror("inotify_add_watch");
    close(inotify_fd);
    exit(EXIT_FAILURE);
}

log_fd = open(LOG_FILE, O_WRONLY | O_CREAT | O_APPEND, 0644);
if (log_fd < 0) {
    perror("open log file");
    close(inotify_fd);
    exit(EXIT_FAILURE);
}

printf("Monitoring dimulai pada direktori: %s\n", dir_path);
printf("Tekan Ctrl+C atau kirim SIGTERM untuk berhenti.\n");

char buffer[EVENT_BUF_LEN];
while (running) {
    int length = read(inotify_fd, buffer, EVENT_BUF_LEN);
    if (length < 0) {
        perror("read");
        break;
    }

    int i = 0;
    while (i < length) {
        struct inotify_event *event = (struct inotify_event *)&buffer[i];

        if (event->len > 0 && has_valid_extension(event->name)) {
            if (event->mask & IN_CREATE)
                log_event("CREATE", event->name, getpid());
            else if (event->mask & IN_MODIFY)
                log_event("MODIFY", event->name, getpid());
            else if (event->mask & IN_DELETE)
                log_event("DELETE", event->name, getpid());
        }

        i += sizeof(struct inotify_event) + event->len;
    }

    close(log_fd);
    inotify_rm_watch(inotify_fd, watch_fd);
    close(inotify_fd);

    printf("Monitoring dihentikan.\n");
    return 0;
}

```

```

root@DESKTOP-J9C60BQ:~# ./file_monitor
=== File Monitor (inotify) ===
Masukkan path direktori untuk dipantau (misal: /tmp/monitor_dir): /tmp/monitor_dir
Monitoring dimulai pada direktori: /tmp/monitor_dir
Tekan Ctrl+C atau kirim SIGTERM untuk berhenti.

```

OUTPUT BEFORE :

- Tidak ada file file_monitor.log (jika belum pernah dijalankan).
- Direktori target (misal /tmp/monitor_dir) tidak sedang dipantau.
- Tidak ada pencatatan otomatis bila ada file yang berubah di direktori tersebut.

```

root@DESKTOP-J9C60BQ:~# ./file_monitor
=== File Monitor (inotify) ===
Masukkan path direktori untuk dipantau (misal: /tmp/monitor_dir): /tmp/monitor_dir
Monitoring dimulai pada direktori: /tmp/monitor_dir
Tekan Ctrl+C atau kirim SIGTERM untuk berhenti.
^C
root@DESKTOP-J9C60BQ:~# cat file_monitor.log
[2025-10-14 22:07:18] [CREATE] [test1.txt] [PID:2921]
[2025-10-14 22:07:18] [MODIFY] [test1.txt] [PID:2921]
[2025-10-14 22:07:46] [MODIFY] [test1.txt] [PID:2921]
[2025-10-14 22:13:17] [MODIFY] [test1.txt] [PID:2947]
[2025-10-14 22:13:17] [MODIFY] [test1.txt] [PID:2947]
[2025-10-14 22:13:52] [CREATE] [a.txt] [PID:2947]
[2025-10-14 22:13:52] [MODIFY] [a.txt] [PID:2947]
[2025-10-14 22:14:08] [MODIFY] [a.txt] [PID:2947]
[2025-10-14 22:14:21] [DELETE] [a.txt] [PID:2947]
[2025-10-14 22:31:33] [CREATE] [a.txt] [PID:2976]
[2025-10-14 22:31:33] [MODIFY] [a.txt] [PID:2976]
[2025-10-14 22:31:46] [MODIFY] [a.txt] [PID:2976]
[2025-10-14 22:32:05] [DELETE] [a.txt] [PID:2976]
root@DESKTOP-J9C60BQ:~#

```

OUTPUT AFTER :

- Program mulai memantau direktori yang dipilih user.
- Jika membuat, mengedit, atau menghapus file .txt atau .log di dalam direktori itu: Sistem langsung mencatat aktivitas tersebut ke dalam file file_monitor.log.

File log akan terus bertambah setiap ada aktivitas baru.

- Saat pengguna menekan Ctrl+C atau mengirim SIGTERM

KESIMPULAN

Program File Monitor berbasis inotify di sistem Linux yang berfungsi untuk memantau perubahan pada file di sebuah direktori secara real-time.

Setiap kali ada file dengan ekstensi .txt atau .log yang dibuat (CREATE), diubah (MODIFY), atau dihapus (DELETE) di dalam direktori yang dipantau, program akan:

1. menampilkan notifikasi aktivitas di terminal (tidak langsung, hanya pemberitahuan awal monitoring),
2. dan mencatat log aktivitas tersebut ke dalam file file_monitor.log

- CUSTOM SHELL

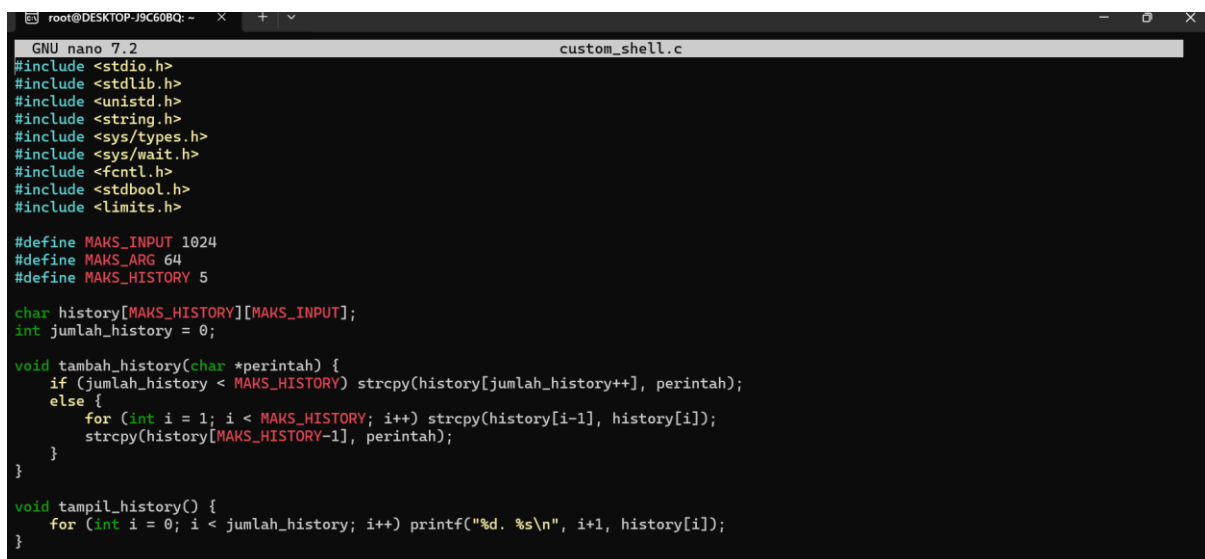
Custom shell adalah program buatan sendiri yang berfungsi seperti command-line interpreter), yaitu program yang membaca, menafsirkan, dan mengeksekusi perintah dari pengguna.

Dengan kata lain, custom shell adalah versi sederhana atau modifikasi dari shell asli yang dibuat sendiri untuk:

- Mempelajari cara kerja shell
- Menambah fitur khusus yang tidak ada di shell standar
- Atau untuk tugas akademik (seperti tugas sistem operasi).

Tujuan Utama Custom Shell

- Memahami Mekanisme Kerja Shell di Sistem Operasi
- Melatih Penggunaan Konsep Sistem Operasi
- Membangun Shell dengan Fitur Khusus
- Menunjukkan Pemahaman tentang Parsing dan Eksekusi Perintah
- Eksperimen dan Riset Pengembangan Sistem



```
GNU nano 7.2 custom_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdbool.h>
#include <limits.h>

#define MAKS_INPUT 1024
#define MAKS_ARG 64
#define MAKS_HISTORY 5

char history[MAKS_HISTORY][MAKS_INPUT];
int jumlah_history = 0;

void tambah_history(char *perintah) {
    if (jumlah_history < MAKS_HISTORY) strcpy(history[jumlah_history++], perintah);
    else {
        for (int i = 1; i < MAKS_HISTORY; i++) strcpy(history[i-1], history[i]);
        strcpy(history[MAKS_HISTORY-1], perintah);
    }
}

void tampil_history() {
    for (int i = 0; i < jumlah_history; i++) printf("%d. %s\n", i+1, history[i]);
}
```

```

void jalankan_tunggal(char **argv, bool background) {
    pid_t pid = fork();
    if (pid == 0) {
        for (int i=0; argv[i]; i++) {
            if (!strcmp(argv[i], ">")) {
                int fd = open(argv[i+1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
                dup2(fd, STDOUT_FILENO);
                close(fd);
                argv[i] = NULL;
            }
            else if (!strcmp(argv[i], ">>")) {
                int fd = open(argv[i+1], O_WRONLY | O_CREAT | O_APPEND, 0644);
                dup2(fd, STDOUT_FILENO);
                close(fd);
                argv[i] = NULL;
            }
            else if (!strcmp(argv[i], "<")) {
                int fd = open(argv[i+1], O_RDONLY);
                dup2(fd, STDIN_FILENO);
                close(fd);
                argv[i] = NULL;
            }
        }

        execvp(argv[0], argv);
        perror("Perintah tidak ditemukan");
        exit(1);
    }
    else if (pid > 0) {

```

```

        if (!background) waitpid(pid, NULL, 0);
    }
    else perror("Gagal fork");
}

void jalankan_pipe(char **kiri, char **kanan) {
    int fd[2];
    pipe(fd);
    pid_t pid1 = fork();
    if (pid1 == 0) {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]); close(fd[1]);
        execvp(kiri[0], kiri);
        perror("Perintah kiri gagal");
        exit(1);
    }
    pid_t pid2 = fork();
    if (pid2 == 0) {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]); close(fd[1]);
        execvp(kanan[0], kanan);
        perror("Perintah kanan gagal");
        exit(1);
    }
    close(fd[0]); close(fd[1]);
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
}

```

```

int main() {
    char input[MAKS_INPUT];
    char *argv[MAKS_ARG];
    char cwd[PATH_MAX];

    while (1) {
        if (getcwd(cwd, sizeof(cwd)) != NULL) printf("%s$ ", cwd);
        else printf("$ ");
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;
        tambah_history(input);

        bool background = false;
        if (input[strlen(input)-1] == '&') {
            background = true;
            input[strlen(input)-1] = '\0';
        }

        char *cmd_kiri[MAKS_ARG], *cmd_kanan[MAKS_ARG];
        char *token = strtok(input, "|");
        if (token) {
            char *kiri = token;
            token = strtok(NULL, "|");
            char *kanan = token;

            if (kanan) {

```

```

    int i=0; char *tok = strtok(kiri, " ");
    while (tok) { cmd_kiri[i++] = tok; tok = strtok(NULL, " "); }
    cmd_kiri[i] = NULL;

    i=0; tok = strtok(kanan, " ");
    while (tok) { cmd_kanan[i++] = tok; tok = strtok(NULL, " "); }
    cmd_kanan[i] = NULL;

    jalankan_pipe(cmd_kiri, cmd_kanan);
    continue;
}
}

int argc=0;
token = strtok(input, " ");
while (token && argc < MAKS_ARG-1) argv[argc++] = token, token = strtok(NULL, " ");
argv[argc] = NULL;
if (!argv[0]) continue;

if (!strcmp(argv[0], "exit")) break;
else if (!strcmp(argv[0], "cd")) {
    if (argv[1]) chdir(argv[1]);
    else printf("Gunakan: cd <direktori>\n");
}
else if (!strcmp(argv[0], "pwd")) {
    if (getcwd(cwd, sizeof(cwd))) printf("%s\n", cwd);
}
else if (!strcmp(argv[0], "echo")) {
    for (int i=1; argv[i]; i++) printf("%s ", argv[i]);
    printf("\n");
}
else if (!strcmp(argv[0], "history")) tampil_history();
else jalankan_tunggal(argv, background);
}
return 0;
}

```

OUTPUT :

```

root@DESKTOP-J9C60BQ:~# gcc -o custom_shell custom_shell.c
root@DESKTOP-J9C60BQ:~# ./custom_shell
/root$ pwd
/root
/root$ ls > daftar.txt
/root$ cat < daftar.txt
custom_shell
custom_shell.c
daftar.txt
file_monitor
file_monitor.c
file_monitor.log
file_ujicoba
multi_procces.s
multi_proces.c
multi_process.c
process_manager
process_manager.c
process_manager.cc
/root$ echo Halo Dunia >> daftar.txt
Halo Dunia >> daftar.txt
/root$ cat daftar.txt
custom_shell
custom_shell.c
daftar.txt
file_monitor
file_monitor.c
file_monitor.log
file_ujicoba
multi_procces.s
multi_proces.c
multi_process.c
process_manager

```

```

process_manager.c
process_manager.cc
/root$ ls | grep c
custom_shell
custom_shell.c
file_monitor.c
file_ujicoba
multi_procces.s
multi_proces.c
multi_process.c
process_manager
process_manager.c
process_manager.cc
/root$ history
1. cat < daftar.txt
2. echo Halo Dunia >> daftar.txt
3. cat daftar.txt
4. ls | grep c
5. history
/root$ exit
root@DESKTOP-J9C60BQ:~# nano custom_shell.c

```

KESIMPULAN :

Program ini adalah Custom Shell sederhana berbasis C, yaitu tiruan ringan dari terminal Linux seperti *bash* atau *sh*.

Shell ini dapat membaca perintah dari pengguna, mengeksekusinya menggunakan `execvp()`, serta mendukung fitur-fitur tambahan seperti:

- Riwayat perintah (history)
- Redirection input/output (>, >>, <)
- Piping sederhana (|)
- Eksekusi background (&)
- Perintah built-in seperti `cd`, `pwd`, `echo`, dan `exit`

- SYTEM INFORMATION TOOL

System Information Tool adalah alat atau program yang digunakan untuk mengumpulkan, menampilkan, dan memantau informasi sistem komputer, seperti spesifikasi perangkat keras (CPU, RAM, disk), status sistem operasi, proses yang berjalan, serta aktivitas jaringan.

Tujuan Utama System Information Tool:

- Memantau Kinerja Sistem
- Mendiagnosis Masalah (Troubleshooting)
- Memberikan Informasi Lengkap Tentang Sistem
- Menjaga Keamanan Sistem
- Mendukung Pemeliharaan dan Optimasi

INPUT :



```
GNU nano 7.2      sistem_info.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>
#include <sys/statvfs.h>
#include <fcntl.h>
#include <ctype.h>

#define BUF_SIZE 4096

void info_cpu() {
    int fd = open("/proc/cpuinfo", O_RDONLY);
    if (fd < 0) { perror("Gagal membuka /proc/cpuinfo"); return; }
    char buf[BUF_SIZE];
    int n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    close(fd);
    char *baris = strtok(buf, "\n");
    char model[256] = "", core[16] = "", freq[32] = "";
    while (baris) {
        if (strcmp(baris, "model name", 10) == 0) strcpy(model, strchr(baris, ':') + 2);
        if (strcmp(baris, "cpu cores", 9) == 0) strcpy(core, strchr(baris, ':') + 2);
        if (strcmp(baris, "cpu MHz", 7) == 0) strcpy(freq, strchr(baris, ':') + 2);
        baris = strtok(NULL, "\n");
    }
    printf("+-----+-----+-----+\n");
    printf("| Informasi CPU          | Nilai          |\n");
    printf("+-----+-----+-----+\n");
```

```

printf("| Model | %s\n", model);
printf("| Jumlah Core | %s\n", core);
printf("| Frekuensi (MHz) | %s\n", freq);
printf("-----+\n");
}

void info_memori() {
    int fd = open("/proc/meminfo", O_RDONLY);
    if (fd < 0) { perror("Gagal membuka /proc/meminfo"); return; }
    char buf[BUF_SIZE];
    int n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    close(fd);
    char *baris = strtok(buf, "\n");
    long total = 0, bebas = 0;
    while (baris) {
        if (strncmp(baris, "MemTotal", 8) == 0) total = atol(strchr(baris, ':') + 1);
        if (strncmp(baris, "MemAvailable", 12) == 0) bebas = atol(strchr(baris, ':') + 1);
        baris = strtok(NULL, "\n");
    }
    long terpakai = total - bebas;
    printf("-----+\n");
    printf("| Informasi Memori | Nilai (KB) | \n");
    printf("-----+\n");
    printf("| Total Memori | %ld\n", total);
    printf("| Memori Bebas | %ld\n", bebas);
    printf("| Memori Terpakai | %ld\n", terpakai);
    printf("-----+\n");
}

```

```

void info_disk() {
    struct statvfs stat;
    if (statvfs("/", &stat) != 0) { perror("Gagal membaca disk"); return; }
    unsigned long total = (stat.f_blocks * stat.f_frsize) / 1024;
    unsigned long bebas = (stat.f_bfree * stat.f_frsize) / 1024;
    unsigned long terpakai = total - bebas;
    printf("-----+\n");
    printf("| Informasi Disk | Nilai (KB) | \n");
    printf("-----+\n");
    printf("| Total Disk | %lu\n", total);
    printf("| Disk Bebas | %lu\n", bebas);
    printf("| Disk Terpakai | %lu\n", terpakai);
    printf("-----+\n");
}

```

```

void info_proses() {
    DIR *d = opendir("/proc");
    if (!d) { perror("Gagal membuka /proc"); return; }
    struct dirent *ent;
    printf("-----+\n");
    printf("| PID | Nama Proses | \n");
    printf("-----+\n");
    while ((ent = readdir(d)) != NULL) {
        if (isdigit(ent->d_name[0])) {
            char path[512];
            snprintf(path, sizeof(path), "/proc/%s/comm", ent->d_name);
            FILE *f = fopen(path, "r");
            if (f) {
                char nama[256];

```

```

                fgets(nama, sizeof(nama), f);
                nama[strcspn(nama, "\n")] = 0;
                printf("| %-25s | %-28s | \n", ent->d_name, nama);
                fclose(f);
            }
        }
    }
    printf("-----+\n");
    closedir(d);
}

void info_network() {
    int fd = open("/proc/net/dev", O_RDONLY);
    if (fd < 0) { perror("Gagal membuka /proc/net/dev"); return; }
    char buf[BUF_SIZE];
    int n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    close(fd);
    printf("-----+\n");
    printf("| Interface | Bytes Diterima | Bytes Dikirim | \n");
    printf("-----+\n");
    char *baris = strtok(buf, "\n");
    int i = 0;
    while (baris) {
        if (i > 1) {
            char iface[64]; unsigned long rcv, snd;
            sscanf(baris, "%[^:]: %lu %u %u %u %u %u %u %u", iface, &rcv, &snd);
            printf("| %-18s | %-16lu | %-16lu | \n", iface, rcv, snd);
        }
        baris = strtok(NULL, "\n");
        i++;
    }
}

```

```

        baris = strtok(NULL, "\n");
        i++;
    }
    printf("-----+\n");
}

void info_load() {
    FILE *f = fopen("/proc/loadavg", "r");
    if (!f) { perror("Gagal membuka /proc/loadavg"); return; }
    double a, b, c;
    fscanf(f, "%lf %lf %lf", &a, &b, &c);
    fclose(f);
    printf("-----+\n");
    printf("| Beban Rata-Rata Sistem | Nilai |");
    printf("-----+\n");
    printf("| 1 Menit | %.2f\n", a);
    printf("| 5 Menit | %.2f\n", b);
    printf("| 15 Menit | %.2f\n", c);
    printf("-----+\n");
}

int main() {
    printf("\n=== Informasi Sistem Linux ===\n\n");
    info_cpu();
    info_memori();
    info_disk();
    info_proses();
    info_network();
    info_load();
}

return 0;
}

```

OUTPUT :

```

root@DESKTOP-J9C60BQ:~# gcc -o sistem_info sistem_info.c
./sistem_info

=== Informasi Sistem Linux ===

-----+
| Informasi CPU | Nilai |
-----+
| Model | Intel(R) Core(TM) i7-10610U CPU @ 1.80GHz |
| Jumlah Core | 4 |
| Frekuensi (MHz) | 2304.006 |
-----+
| Informasi Memori | Nilai (KB) |
-----+
| Total Memori | 7971036 |
| Memori Bebas | 7455132 |
| Memori Terpakai | 515904 |
-----+
| Informasi Disk | Nilai (KB) |
-----+
| Total Disk | 1055762868 |
| Disk Bebas | 1053925368 |
| Disk Terpakai | 1837500 |
-----+
| PID | Nama Proses |
-----+
| 1 | systemd |
| 2 | init-systemd(Ub |
| 7 | init |
| 169 | cron |

```

170	dbus-daemon	
177	systemd-logind	
180	wsl-pro-service	
196	agetty	
201	agetty	
210	unattended-upgr	
300	login	
343	systemd	
344	(sd-pam)	
368	bash	
636	polkitd	
1591	systemd-journal	
1639	systemd-timesyn	
1691	systemd-udev	
1742	systemd-resolve	
2050	rsyslogd	
5166	SessionLeader	
5167	Relay(5173)	
5173	bash	
5358	sistem_info	

Interface	Bytes Diterima	Bytes Dikirim
lo	5259	5259
eth0	10366257	213548

Beban Rata-Rata Sistem	Nilai
1 Menit	0.00
5 Menit	0.00
15 Menit	0.00

KESIMPULAN :

Program C di atas adalah alat pemantau informasi sistem Linux (System Information Tool) yang menampilkan berbagai data real-time dari direktori /proc dan sistem file. Ketika dijalankan, program akan mencetak tabel-tabel berisi informasi utama tentang sistem, meliputi:

1. Informasi CPU

- Model CPU (nama prosesor)
- Jumlah core
- Frekuensi (MHz)

2. Informasi Memori

- Total memori fisik
- Memori bebas (tersedia)
- Memori terpakai (hasil pengurangan total–bebas)

3. Informasi Disk

- Total kapasitas disk
- Ruang bebas
- Ruang terpakai

4. Daftar Proses

- PID (Process ID)
- Nama proses yang sedang aktif

5. Informasi Jaringan

- Nama interface jaringan (misalnya eth0, lo, wlan0)
- Jumlah byte diterima dan byte dikirim

6. Beban Sistem (Load Average)

- Rata-rata beban CPU selama 1, 5, dan 15 menit terakhir